

README – Proyecto Web Quixo

GRUPO 4

Programación Avanzada Web

8 de diciembre de 2025

1. Información general del proyecto

Número de grupo: 4

Nombre del curso: Programación Avanzada Web

Nombre de la aplicación: Proyecto Web Quixo

Breve descripción del proyecto:

La aplicación web desarrollada implementa el juego de mesa **Quixo**, una evolución del clásico Tic-tac-toe (“gato”). Permite jugar en modo **dos jugadores (2P)** y **cuatro jugadores (4P)**, visualizar y exportar partidas finalizadas, así como consultar estadísticas de jugadores y equipos.

2. Integrantes del grupo

A continuación se muestran los integrantes del grupo con nombre completo y carné:

Nombre completo	Carné
Sebastián Mora Figueroa	FI22027357
Kevin Sánchez Seas	FI22025309
Fiorella Portuguez Rojas	FI22024660

3. Información de Git de los integrantes

En la siguiente tabla se detalla la información de Git de cada integrante (usuario y correo de Git):

Nombre	Usuario Git	Correo Git
Sebastián Mora Figueroa	samorafí	sebas Mora172522@gmail.com
Kevin Sánchez Seas	kasanche11	ksanchez90688@ufide.ac.cr
Fiorella Portuguez Rojas	fioportu	portuguezfiore@gmail.com

4. Tecnologías, tipo de aplicación y arquitectura

4.1. Frameworks y herramientas utilizadas

A continuación se listan los frameworks y herramientas principales utilizados:

- **Tipo de aplicación:** SPA (Single Page Application).
- **Frontend:** React (Vite).

- **Backend:** ASP.NET Core (Web API).
- **ORM:** Entity Framework Core.
- **Base de datos:** SQL Server.
- **Cliente HTTP:** Axios (frontend) para consumir la API.
- **Enrutamiento en frontend:** React Router DOM.
- **Editor/IDE:** Visual Studio Code
- **Control de versiones:** GitHub.

4.2. Tipo de aplicación: SPA

La aplicación es una **SPA (Single Page Application)** porque:

- El frontend en React se carga una sola vez en el navegador.
- La navegación entre vistas (menú principal, juego, historial, estadísticas) se realiza de forma dinámica en el cliente, usando React Router, sin recargar completamente la página.
- La comunicación con el servidor se realiza mediante peticiones a una Web API (REST) para obtener y enviar datos (jugadores, partidas, estadísticas).

4.3. Arquitectura utilizada: Web API + SPA

La arquitectura general combina:

- **Backend ASP.NET Core Web API** con una estructura por capas (controladores, capa de dominio/servicios, acceso a datos con EF Core).
- **Frontend SPA en React** que consume los endpoints de la Web API.
- **Base de datos SQL Server** para persistir jugadores, partidas, historial y estadísticas.

Razones para utilizar esta arquitectura:

- Separación clara de responsabilidades entre backend (lógica de negocio, persistencia) y frontend (interfaz de usuario).
- Facilidad para exponer servicios REST que podrían ser consumidos por otras interfaces en el futuro.
- React y ASP.NET Core son tecnologías ampliamente utilizadas, con buena documentación y soporte.

5. Definición de la base de datos

La aplicación utiliza una base de datos relacional en **SQL Server**. A continuación se muestra el script simplificado de las tablas principales:

5.1. Script de creación de tablas

— *TABLA: Jugadores*

```
CREATE TABLE dbo.Jugadores (  
    JugadorId      INT IDENTITY(1,1) NOT NULL PRIMARY KEY,  
    Nombre         NVARCHAR(100)      NOT NULL,  
    FechaCreacion  DATETIME2(3)      NOT NULL DEFAULT SYSUTCDATETIME()  
);
```

— *TABLA: Partidas*

```
CREATE TABLE dbo.Partidas (  
    PartidaId      INT IDENTITY(1,1) NOT NULL PRIMARY KEY,  
    Modo           CHAR(2)           NOT NULL  
    CONSTRAINT CK_Partidas_Modo CHECK (Modo IN ( '2P', '4P' )),  
    FechaCreacion  DATETIME2(0)      NOT NULL DEFAULT SYSDATETIME() ,  
    FechaSobrescrita DATETIME2(0)      NULL,  
    FechaFinalizada DATETIME2(0)      NULL,  
    DuracionSegundos INT              NULL,  
    JugadorOid     INT                NULL,  
    JugadorXid     INT                NULL,  
    EquipoA1Id     INT                NULL,  
    EquipoA2Id     INT                NULL,  
    EquipoB1Id     INT                NULL,  
    EquipoB2Id     INT                NULL,  
    GanadorSimbolo CHAR(1)            NULL, — 'O' o 'X' (solo 2P)  
    GanadorEquipo  CHAR(1)            NULL, — 'A' o 'B' (solo 4P)  
    TableroFinalXml XML              NULL,  
    HistorialXml    XML              NOT NULL  
);
```

— *TABLA: Equipos4P*

```
CREATE TABLE dbo.Equipos4P (  
    EquipoId       INT IDENTITY(1,1) NOT NULL PRIMARY KEY,  
    Jugador1Id     INT                NOT NULL,  
    Jugador2Id     INT                NOT NULL,  
    FechaCreacion  DATETIME2(3)      NOT NULL DEFAULT SYSUTCDATETIME()  
);
```

— *TABLA: PartidasEquipos4P*

```
CREATE TABLE dbo.PartidasEquipos4P (  
    PartidaEquipoId INT IDENTITY(1,1) NOT NULL PRIMARY KEY,  
    PartidaId       INT                NOT NULL, — Modo = '4P'  
    EquipoId        INT                NOT NULL, — Referencia a Equipos4P  
    Letra           CHAR(1)            NOT NULL  
    CONSTRAINT CK_PartidasEquipos4P_Letra CHECK (Letra IN ( 'A', 'B' )),  
    Simbolo         CHAR(1)            NOT NULL  
    CONSTRAINT CK_PartidasEquipos4P_Simbolo CHECK (Simbolo IN ( 'O', 'X' ))  
);
```

— *TABLA: PartidaParticipantes*

```
CREATE TABLE dbo.PartidaParticipantes (  
    PartidaParticipanteId INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
```

```

PartidaId          INT          NOT NULL,
JugadorId          INT          NOT NULL,
Rol                NVARCHAR(3)  NOT NULL,
Equipo             CHAR(1)      NULL
CONSTRAINT CK_PartidaParticipantes_Equipo
CHECK (Equipo IN ('A', 'B') OR Equipo IS NULL),
OrdenTurno         SMALLINT     NULL
);

```

— *TABLA: EstadisticasJugadores2P*

```

CREATE TABLE dbo.EstadisticasJugadores2P (
    JugadorId          INT          NOT NULL PRIMARY KEY,
    Jugadas            INT          NOT NULL DEFAULT 0,
    Ganadas            INT          NOT NULL DEFAULT 0,
    EfectividadPorc    DECIMAL(5,2) NULL
);

```

— *TABLA: EstadisticasEquipos4P*

```

CREATE TABLE dbo.EstadisticasEquipos4P (
    EquipoId          INT          NOT NULL PRIMARY KEY,
    Jugadas            INT          NOT NULL DEFAULT 0,
    Ganadas            INT          NOT NULL DEFAULT 0,
    EfectividadPorc    DECIMAL(5,2) NULL
);

```

5.2. Descripción de entidades

- **Jugadores:** almacena los datos básicos de cada jugador (identificador y nombre).
- **Partidas:** guarda la información general de cada partida (modo 2P o 4P, fechas, duración, participantes, ganador, tablero final e historial completo en XML).
- **Equipos4P:** define las parejas reutilizables para el modo de cuatro jugadores.
- **PartidasEquipos4P:** relaciona qué equipos (A y B) jugaron una partida 4P y qué símbolo usaron (O o X).
- **PartidaParticipantes:** lista detallada de los participantes en cada partida, unificando los roles:
 - Modo 2P: “O” y “X”.
 - Modo 4P: “A1”, “A2”, “B1”, “B2”, con su respectivo equipo A o B y orden de turno.
- **EstadisticasJugadores2P:** estadísticas agregadas para modo de 2 jugadores (jugadas, ganadas, efectividad).
- **EstadisticasEquipos4P:** estadísticas agregadas para equipos en modo 4 jugadores.

5.3. Diagrama de base de datos

El diagrama completo de la base de datos puede consultarse en el siguiente enlace:

- https://github.com/samorafi/Proyecto_PrograAvanzadaWeb/blob/main/DB/DBQuixo.png

6. Instructivo de instalación, compilación y ejecución

6.1. Backend: ASP.NET Core Web API

6.1.1. Requisitos previos

- .NET SDK (versión utilizada en el proyecto .NET 8.0).
- SQL Server instalado y con una base de datos accesible para `Quixodb`.
- Herramienta para ejecutar scripts SQL (SQL Server Management Studio).

6.1.2. Crear solución y proyecto

```
# Carpeta de trabajo
mkdir quixo && cd quixo

# Solucion y proyecto Web API
dotnet new sln -n Quixo
dotnet new webapi -n Quixo.Api

# Agregar proyecto a la solucion
dotnet sln Quixo.sln add Quixo.Api/Quixo.Api.csproj
```

6.1.3. Instalación de paquetes NuGet

```
cd Quixo.Api

# EF Core para SQL Server y herramientas de migraciones
dotnet add package Microsoft.EntityFrameworkCore.SqlServer
dotnet add package Microsoft.EntityFrameworkCore.Tools
dotnet add package Microsoft.EntityFrameworkCore
```

6.1.4. Configuración de la base de datos

1. Crear la base de datos `Quixodb` en SQL Server.
2. Ejecutar el script de creación de tablas mostrado en la Sección 4.
3. Configurar la cadena de conexión en `appsettings.json`:

```
"ConnectionStrings": {
  "SqlServer": "Server=SERVIDOR; Database=Quixodb; Trusted_Connection=True; TrustSe
}
```

6.1.5. Compilación y ejecución del backend

```
# Desde la carpeta Quixo.Api
dotnet build
dotnet run
```

El backend levantará la Web API (`http://localhost:5207`).

6.2. Frontend: React (SPA con Vite)

6.2.1. Creación e instalación inicial

```
npm create vite@latest quixo-front -- --template react
# Framework: React
# Variant: JavaScript
# rolldown-vite: No
```

```
cd quixo-front
npm install
npm i axios react-router-dom classnames
```

6.2.2. Configuración

- Configurar la URL base de la API (<http://localhost:5207>) en el archivo de servicios de Axios.
- Ajustar rutas de React Router para:
 - Menú principal (jugar nueva partida, ver partidas finalizadas, ver estadísticas).
 - Vista de juego (tablero 5x5, reloj de 7 segmentos, botón de reinicio).
 - Vista de historial de partidas.
 - Vista de estadísticas de jugadores y equipos.

6.2.3. Compilación y ejecución del frontend

```
# Desarrollo
npm run dev
```

```
# Compilacion para produccion
npm run build
```

La aplicación esta disponible en <http://localhost:5173>.

7. Referencias (sitios web y material consultado)

A continuación se listan las referencias de sitios web utilizados para entender reglas, conceptos y ejemplos:

- **Cómo jugar Quixo:**
<https://maldon.com.ar/blog/projects/quixo/>
- **QUIXO - ¿Cómo se juega? - Tutorial en español y partida:**
<https://www.youtube.com/watch?v=IHtf0YiNuEo>
- **SPA vs MPA y las arquitecturas Web:**
<https://www.arquitecturajava.com/spa-vs-mpa-y-las-arquitecturas-web/>
- **Tutorial: Creación de aplicaciones ASP.NET Core con React:**
<https://learn.microsoft.com/es-es/visualstudio/javascript/tutorial-asp-net-core-with-react-view=visualstudio>

- **Display de 7 segmentos (conceptos y funcionamiento):**
<https://industriasgsl.com/blogs/automatizacion/display-de-7-segmentos>
- **Language Integrated Query (LINQ):**
<https://learn.microsoft.com/es-mx/dotnet/csharp/linq/>
- **Habilitar solicitudes entre orígenes (CORS) en ASP.NET Core:**
<https://learn.microsoft.com/es-mx/aspnet/core/security/cors?view=aspnetcore-10.0>
- **Fetch API y llamadas al backend:**
https://developer.mozilla.org/es/docs/Web/API/Fetch_API/Using_Fetch
- **Timers en React y técnicas recomendadas:**
<https://react.dev/learn/keeping-components-pure>
- **Crear un documento en LaTeX:**
<https://manualdelatex.com/tutoriales/crear-un-documento>

8. Uso de agentes de IA y prompts

En el desarrollo del proyecto se utilizaron agentes de IA para apoyo en el diseño de componentes específicos. A continuación se listan los prompts y recursos principales:

8.1. Prompts y enlaces

- **Cómo hacer un tablero 5x5 con React (X, O y neutro):**
Prompt y conversación: <https://chat.deepseek.com/share/0uqkmm2gqnp4z6zftn>
- **Reloj digital de 7 segmentos en React:**
Prompt y conversación: <https://chat.deepseek.com/share/0uqkmm2gqnp4z6zftn>
- **Fórmula de efectividad (porcentaje de partidas ganadas):**
Prompt y conversación: <https://chat.deepseek.com/share/0uqkmm2gqnp4z6zftn>
- **Generar tablas con migraciones**
Prompt y conversación: <https://chatgpt.com/share/69373796-790c-8002-93a5-3536469bd2bd>
- **Modificar lógica para 2P y 4P**
Prompt y conversación: <https://chatgpt.com/share/69373ac8-964c-8002-8fe2-3491c0959d94>